How to Train Machine Learning Models: An End-to-End Practical and Academic Guide

A workflow guide for career-transition learners, business analysts, aspiring AI/ML practitioners, and trainers

OpenAI

31 March 2026

Contents

1 Preface **5**
1.1 How to read this guide . . . 5
1.2 What "high quality" means in this guide . . . 5

2 Executive Summary **6**

3 Visual Overview of the Workflow **6**

4 Problem **9**
4.1 Definition . . . 9
4.2 What to do . . . 9
4.2.1 1. Write the decision statement . . . 9
4.2.2 2. Define the target exactly . . . 9
4.2.3 3. Define success metrics at business and model levels . . . 9
4.2.4 4. Check whether machine learning is actually needed . . . 9
4.2.5 5. Identify constraints . . . 9
4.3 Minimum-quality standard . . . 10
4.4 Typical failure modes . . . 10
4.5 Recommended tools and official references . . . 10

5 Data **10**
5.1 Definition . . . 10
5.2 The major data families . . . 10
5.2.1 Structured data . . . 10
5.2.2 Semi-structured data . . . 10
5.2.3 Unstructured data . . . 10
5.2.4 Specialized data . . . 11
5.3 What to do . . . 11
5.3.1 1. Create a data inventory . . . 11
5.3.2 2. Define the row grain . . . 11
5.3.3 3. Define the label source . . . 11
5.3.4 4. Prevent leakage at the data-design stage . . . 11
5.3.5 5. Create a data dictionary . . . 11
5.4 Data-type-specific guidance . . . 11
5.5 Choosing the learning paradigm . . . 12
5.5.1 Supervised learning . . . 12
5.5.2 Unsupervised learning . . . 12
5.5.3 Reinforcement learning . . . 12
5.6 Minimum-quality standard . . . 12
5.7 Official references . . . 12

6 Clean **13**
6.1 Definition . . . 13

6.2 What to do . . . 13
6.2.1 1. Resolve schema issues . . . 13
6.2.2 2. Handle missing data deliberately . . . 13
6.2.3 3. Detect duplicates . . . 13
6.2.4 4. Standardize categorical values . . . 13
6.2.5 5. Treat outliers logically . . . 13
6.2.6 6. Build a reproducible cleaning pipeline . . . 13
6.3 Preprocessing operations by data type . . . 13
6.4 Minimum-quality standard . . . 14
6.5 Common limitations and warnings . . . 14
6.6 Helpful official references . . . 14

7 Explore **14**
7.1 Definition . . . 14
7.2 What to do . . . 14
7.2.1 1. Profile the target . . . 14
7.2.2 2. Profile each feature . . . 14
7.2.3 3. Look for interaction and segmentation effects . . . 15
7.2.4 4. Use the right visual summaries . . . 15
7.2.5 5. Write exploratory findings as hypotheses . . . 15
7.3 Minimum-quality standard . . . 15
7.4 Real-life metaphor . . . 15
7.5 Recommended tools . . . 15

8 Features **15**
8.1 Definition . . . 15
8.2 Core principle . . . 15
8.3 Common feature types . . . 15
8.3.1 Numeric transforms . . . 15
8.3.2 Encoded categoricals . . . 16
8.3.3 Time-derived features . . . 16
8.3.4 Text features . . . 16
8.3.5 Image and audio features . . . 16
8.4 How to identify predictive features . . . 16
8.5 Feature selection and dimensionality reduction . . . 16
8.5.1 Feature selection . . . 16
8.5.2 Dimensionality reduction . . . 16
8.6 Encoding and scaling rules of thumb . . . 16
8.7 Regularization . . . 17
8.7.1 L2 regularization . . . 17
8.7.2 L1 regularization . . . 17
8.7.3 Dropout and weight decay . . . 17
8.8 Minimum-quality standard . . . 17
8.9 Important limitations . . . 17
8.10 Official references . . . 17

9 Train **18**
9.1 Definition . . . 18
9.2 Split design comes before algorithm choice . . . 18
9.2.1 Common split strategies . . . 18
9.3 The three major learning paradigms . . . 18
9.3.1 Supervised learning . . . 18
9.3.2 Unsupervised learning . . . 18
9.3.3 Reinforcement learning . . . 18
9.4 Supervised algorithms: what to use, when, and why . . . 18
9.4.1 Linear regression / logistic regression . . . 18
9.4.2 Decision trees . . . 19
9.4.3 Random forests . . . 19
9.4.4 Gradient boosting (general), XGBoost, LightGBM, CatBoost . . . 19
9.4.5 Support vector machines . . . 19
9.4.6 k-nearest neighbors . . . 19
9.4.7 Neural networks / deep learning . . . 19

9.5 Unsupervised algorithms . . . 20
9.5.1 k-means . . . 20
9.5.2 DBSCAN . . . 20
9.5.3 Hierarchical clustering . . . 20
9.5.4 PCA / Truncated SVD . . . 20
9.5.5 Isolation Forest / Local Outlier Factor / one-class methods . . . 20
9.6 Reinforcement learning algorithms . . . 20
9.6.1 DQN . . . 20
9.6.2 PPO . . . 20
9.6.3 SAC / DDPG / TD3 . . . 20
9.6.4 RL limitations that beginners often underestimate . . . 20
9.7 Comparative training table . . . 21
9.8 Minimum-quality standard . . . 21
9.9 Official references . . . 21

10 Evaluate **21**
10.1 Definition . . . 21
10.2 Classification metrics . . . 22
10.2.1 Confusion-matrix terms . . . 22
10.2.2 Precision . . . 22
10.2.3 Recall . . . 22
10.2.4 F_1 . . . 22
10.2.5 Accuracy . . . 22
10.2.6 ROC-AUC and PR-AUC . . . 22
10.2.7 Log loss / cross-entropy . . . 22
10.3 Regression metrics . . . 22
10.3.1 Mean Absolute Error (MAE) . . . 22
10.3.2 Mean Squared Error (MSE) . . . 22
10.3.3 Root Mean Squared Error (RMSE) . . . 22
10.3.4 Coefficient of determination R^2 . . . 23
10.4 Clustering and unsupervised evaluation . . . 23
10.5 Reinforcement-learning evaluation . . . 23
10.6 Threshold selection . . . 23
10.7 Calibration . . . 23
10.8 Error analysis . . . 23
10.9 Minimum-quality standard . . . 23
10.10 Official references . . . 23

11 Tune **24**
11.1 Definition . . . 24
11.2 What to tune . . . 24
11.2.1 Classical ML . . . 24
11.2.2 Deep learning . . . 24
11.2.3 Reinforcement learning . . . 24
11.3 Search methods . . . 24
11.3.1 Grid search . . . 24
11.3.2 Random search . . . 24
11.3.3 Bayesian optimization / model-based search . . . 24
11.3.4 Early-stopping schedulers . . . 24
11.4 Practical tuning rules . . . 24
11.5 Tuning by algorithm family . . . 25
11.6 Minimum-quality standard . . . 25
11.7 Official references . . . 25

12 Deploy **25**
12.1 Definition . . . 25
12.2 Deployment patterns . . . 25
12.2.1 Batch scoring . . . 25
12.2.2 Online API inference . . . 26
12.2.3 Embedded / edge inference . . . 26
12.3 What to package . . . 26
12.4 Serialization and portability . . . 26

12.4.1 Practical rule . . . 26
12.5 Serving stack . . . 26
12.6 Deployment checklist . . . 26
12.7 Common deployment mistakes . . . 26
12.8 Clinic example . . . 26

13 Monitor **27**
13.1 Definition . . . 27
13.2 The four monitoring layers . . . 27
13.2.1 1. System monitoring . . . 27
13.2.2 2. Data-quality monitoring . . . 27
13.2.3 3. Drift monitoring . . . 27
13.2.4 4. Performance monitoring . . . 27
13.3 What to compare against . . . 27
13.4 Alert design . . . 27
13.5 Retraining triggers . . . 27
13.6 Minimum-quality standard . . . 27
13.7 Practical limitation . . . 28

14 Repeat **28**
14.1 Definition . . . 28
14.2 What to revisit . . . 28
14.3 Post-deployment review questions . . . 28
14.4 Improvement paths . . . 28

15 A Minimum Standard Workflow Blueprint **28**
15.1 Phase 1: Scope correctly . . . 28
15.2 Phase 2: Build trustworthy data . . . 28
15.3 Phase 3: Prepare the learning substrate . . . 29
15.4 Phase 4: Model systematically . . . 29
15.5 Phase 5: Productionize responsibly . . . 29

16 Algorithm and Technique Selection Cheat Sheets **29**
16.1 Supervised learning cheat sheet . . . 29
16.2 Unsupervised learning cheat sheet . . . 29
16.3 Reinforcement learning cheat sheet . . . 30

17 Teaching Notes for Trainers **30**
17.1 How to explain the workflow simply . . . 30
17.2 Suggested classroom sequence . . . 30

18 Conclusion **30**

19 Appendix A. Exhaustive Glossary of Technical Terms **32**
19.1 A . . . 32
19.2 B . . . 32
19.3 C . . . 32
19.4 D . . . 33
19.5 E . . . 33
19.6 F . . . 33
19.7 G . . . 33
19.8 H . . . 34
19.9 I . . . 34
19.10K . . . 34
19.11L . . . 34
19.12M . . . 34
19.13N . . . 35
19.14O . . . 35
19.15P . . . 35
19.16Q . . . 35
19.17R . . . 35
19.18S . . . 36
19.19T . . . 36

19.20U . . . 36
19.21V . . . 37
19.22W . . . 37

20 Appendix B. Python Code Templates by Workflow Section 38
20.1 B1. Problem framing skeleton . . . 38
20.2 B2. Data audit with pandas . . . 38
20.3 B3. Reproducible preprocessing pipeline for structured data . . . 38
20.4 B4. Strong tabular baseline with gradient boosting . . . 39
20.5 B5. Cross-validation and evaluation . . . 39
20.6 B6. Threshold tuning . . . 39
20.7 B7. Hyperparameter tuning with scikit-learn . . . 40
20.8 B8. Experiment tracking with MLflow . . . 40
20.9 B9. Export model to ONNX (illustrative) . . . 40
20.10B10. Minimal FastAPI serving skeleton . . . 41
20.11B11. Data-quality validation idea . . . 41
20.12B12. Drift monitoring idea with Evidently . . . 41
20.13B13. Text classification baseline . . . 42
20.14B14. Keras structured-data baseline . . . 42
20.15B15. PyTorch image-classification skeleton . . . 42
20.16B16. Stable-Baselines3 PPO example . . . 43
20.17B17. A compact end-to-end checklist object . . . 43

21 References 44

1 Preface

This guide explains the full machine learning workflow from the first business question to production monitoring and retraining. It is written for readers who may be changing careers into technology, teaching machine learning, or building practical capability from a business or analytics background. The structure follows a standard lifecycle:

Problem → Data → Clean → Explore → Features → Train → Evaluate → Tune → Deploy → Monitor → Repeat

The document uses a **single running metaphor** throughout: **a clinic that wants to predict which patients are likely to miss an appointment**. Think of the model as a triage assistant. It does not replace staff judgment; it helps the clinic decide where to send reminders, callbacks, or transport support. This example is concrete enough for beginners and realistic enough for practitioners.

1.1 How to read this guide

- If you are new to machine learning, read the sections in order.
- If you already work in analytics, use the checklists and comparison tables as a minimum-quality standard.
- If you teach, use the metaphor, glossary, and code appendix as reusable training material.
- If you build production systems, pay special attention to **data leakage, validation design, deployment, and monitoring**.

1.2 What "high quality" means in this guide

A high-quality machine learning product should satisfy five conditions:

1. It solves a real decision problem.
2. Its data and preprocessing are reproducible.
3. Its validation design matches the real use case.
4. Its deployment path is operationally sound.
5. Its monitoring detects when the model or data stop behaving as expected.

2 Executive Summary

Machine learning is best understood as a disciplined decision system rather than as a bag of algorithms. In practice, most project failures occur **before** model fitting or **after** model release: bad problem framing, weak data contracts, leakage, misaligned metrics, and missing monitoring are more damaging than an imperfect choice between algorithms. The official documentation of scikit-learn, pandas, TensorFlow/Keras, PyTorch, MLflow, ONNX, Docker, Kubernetes, Great Expectations, and Evidently collectively supports this full-lifecycle view [R1–R24].

The core idea is simple:

- **Problem** defines what success means.
- **Data** determines what the model can possibly learn.
- **Cleaning and exploration** expose risk before it becomes hidden error.
- **Features and training** convert raw signals into learnable patterns.
- **Evaluation and tuning** decide whether the model is genuinely useful.
- **Deployment and monitoring** determine whether the model remains useful in the real world.

A useful beginner mental model is this:

> Training a model is like training a new staff member for the clinic.
> The business problem is the job description.
> The data is the case history.
> Cleaning is correcting incomplete paperwork.
> Feature engineering is turning raw notes into actionable cues.
> Training is practice.
> Evaluation is an exam with unseen cases.
> Deployment is putting the person on shift.
> Monitoring is supervision after onboarding.

3 Visual Overview of the Workflow

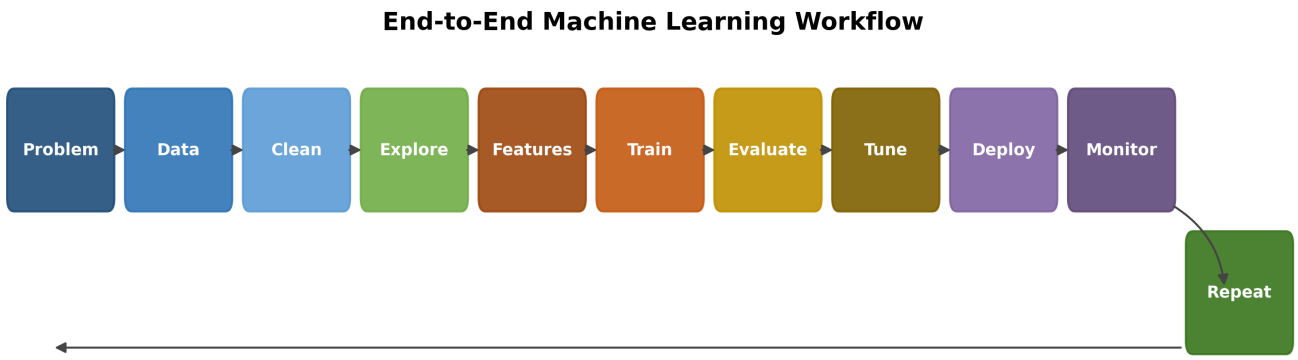

Figure 1: Workflow overview.

Legend

- Blue - Scoping and data understanding
- Orange - Model building and selection
- Purple - Productionization and observability
- Green - Iteration and improvement

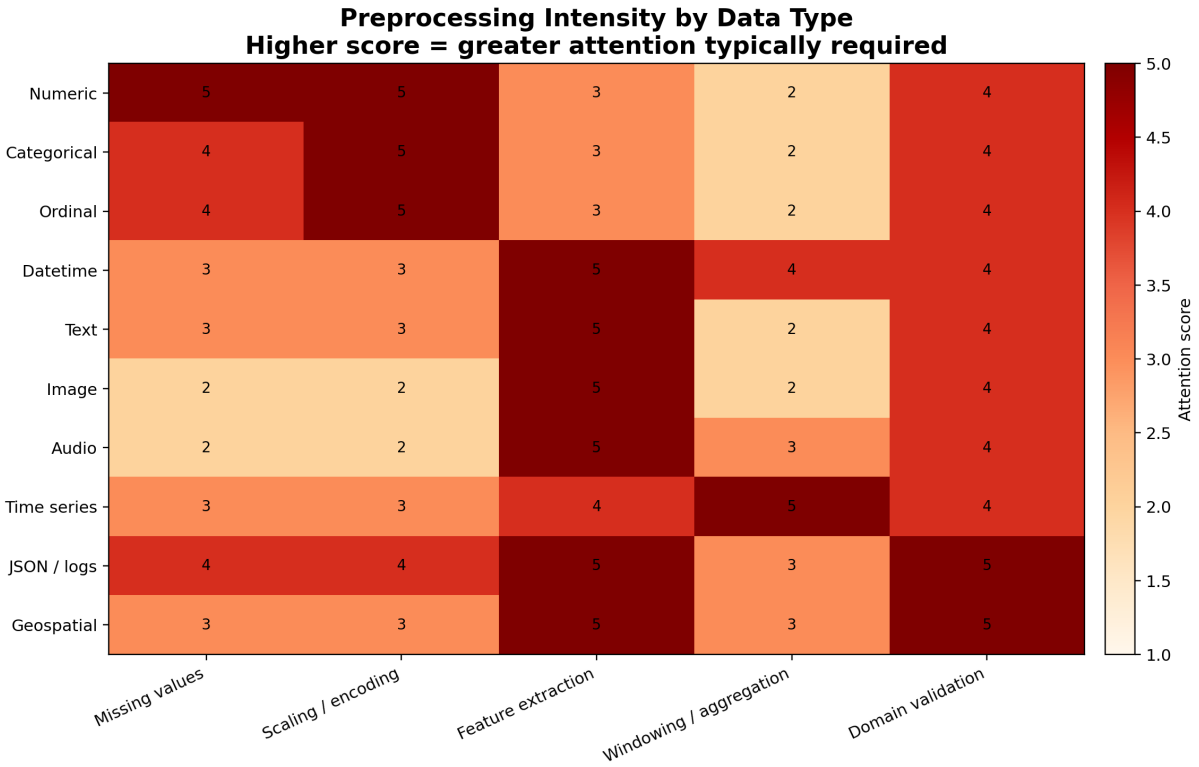

Figure 2: Preprocessing intensity by data type.

Legend

- Higher score means that area usually deserves more deliberate attention in practice.

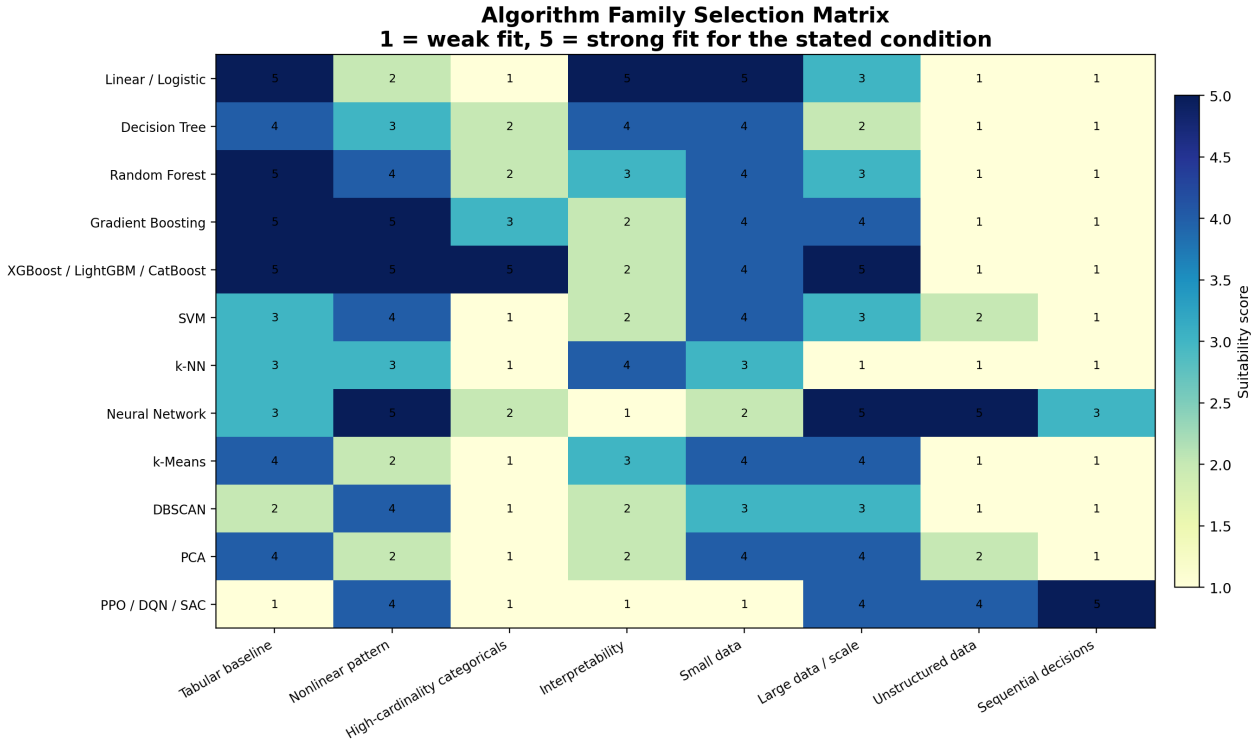

Figure 3: Algorithm family selection matrix.

Legend

- 1 = weak fit
- 5 = strong fit
- Use this as a starting heuristic, not as a law.

4 Problem

4.1 Definition

A machine learning problem is a decision problem expressed precisely enough that data can be used to estimate a mapping from inputs to outputs. In simple terms, the model learns a rule:

$$\hat{y} = f(X)$$

where X is the set of input features and $\hat{y}$ is the predicted output.

For the clinic example:

- **Inputs**: patient age band, day of week, lead time, appointment type, prior no-show count, distance to clinic, reminder history.
- **Output**: whether the patient is likely to miss the appointment.

4.2 What to do

4.2.1 1. Write the decision statement

Use a sentence of the form:

> "We want to predict [**target**] for [**unit of prediction**] at [**decision time**] so that [**action**] can be taken."

Example:

> "We want to predict whether a scheduled appointment will be missed for each patient visit, 24 hours before the visit, so that staff can prioritize reminders and interventions."

4.2.2 2. Define the target exactly

A target must be operational, not vague.

Bad target: - "Patient engagement"

Good target: - "Appointment marked as no-show in the scheduling system within the day of the visit"

4.2.3 3. Define success metrics at business and model levels

Business metrics: - reduced no-show rate - improved staff productivity - cost saved per additional kept appointment

Model metrics: - recall at a chosen alert threshold - precision for flagged patients - calibration quality if probability estimates drive actions

4.2.4 4. Check whether machine learning is actually needed

Not every problem needs ML. Use rules first if: - the pattern is deterministic, - the business rule is stable, - the regulation demands explicit rule logic, - the available data is tiny.

A good minimum baseline is often: - historical average, - a simple rule, - linear or logistic regression, - a shallow tree.

4.2.5 5. Identify constraints

Include: - latency budget, - fairness constraints, - data privacy, - interpretability requirements, - retraining frequency, - integration limits, - cost of false positives and false negatives.

4.3 Minimum-quality standard

A project should not move forward until all of the following are written down:

Item	Minimum standard
Decision statement	One-sentence operational definition
Target	Exact label logic and timestamp
Prediction unit	What one row means
Decision time	When the prediction is made
Action	What happens after prediction
Success metrics	At least one business metric and one model metric
Constraints	Privacy, fairness, latency, interpretability, budget

4.4 Typical failure modes

- Target defined after looking at the data
- Predicting something that is already known at decision time
- Treating correlation as deployable action
- Optimizing an easy metric instead of the real business outcome
- Ignoring class imbalance and action thresholds

4.5 Recommended tools and official references

- scikit-learn user guide for task framing, pipelines, model selection, metrics, and inspection [R1–R4]
- TensorFlow/Keras structured data tutorials for end-to-end supervised tabular workflows [R8–R10]
- PyTorch tutorials for deep learning workflows where the data is unstructured or custom training loops are needed [R11–R12]

5 Data

5.1 Definition

Data is the observable evidence from which the model learns. The official pandas documentation emphasizes careful handling of data types, missing values, categoricals, and time-like values because type semantics determine downstream correctness [R5–R7].

5.2 The major data families

5.2.1 Structured data

Highly organized, row-column data with fixed schema.

Examples: - spreadsheets - SQL tables - CRM exports - sensor tables

Common subtypes: - numeric - binary - categorical - ordinal - datetime - geospatial coordinates

5.2.2 Semi-structured data

Data with some organization but flexible schema.

Examples: - JSON API payloads - event logs - XML - key-value records

5.2.3 Unstructured data

Data not naturally organized into fixed columns.

Examples: - free text - documents - images - audio - video

5.2.4 Specialized data

Examples: - time series - graph/network data - embeddings - reinforcement learning trajectories

5.3 What to do

5.3.1 1. Create a data inventory

For every source, record: - system of origin - owner - refresh cadence - grain - schema - join keys - missingness profile - access permissions - expected historical depth

5.3.2 2. Define the row grain

A row must represent one consistent thing. Examples: - one customer - one transaction - one appointment - one image - one sentence - one time window - one state-action-reward transition

In the clinic example, one row should represent **one appointment**.

5.3.3 3. Define the label source

Specify: - where labels come from, - when they become known, - whether they are noisy, - whether manual review is required.

5.3.4 4. Prevent leakage at the data-design stage

Leakage means training on information unavailable at prediction time. Common examples: - post-outcome statuses, - future timestamps, - manually corrected records created after the event, - aggregations that accidentally include the future.

5.3.5 5. Create a data dictionary

At minimum, the dictionary should include: - column name - type - definition - allowed values - missingness meaning - transformation notes - leakage risk - owner

5.4 Data-type-specific guidance

Data type	Common examples	Minimum preprocessing logic	Good first model family	Key caution
Numeric	age, price, duration	missing-value handling, scaling if needed, outlier review	linear models, trees, boosting, neural nets	units and extreme values
Binary	yes/no flags	cast consistently, verify semantics of 0/1	logistic regression, trees, boosting	hidden missingness
Categorical nominal	city, product, specialty	one-hot, target-aware encoding with care, or native categorical support	trees/boosting, linear with one-hot	high cardinality, leakage in target encoding
Categorical ordinal	risk band, education level	preserve order; encode ordinally	linear, trees, boosting	false ordering if labels are arbitrary
Datetime	booking time, event date	parse, time-zone align, derive durations/cycles	trees, boosting, time models	future information leakage

Data type	Common examples	Minimum preprocessing logic	Good first model family	Key caution
Text	notes, emails	normalize, tokenize, vectorize or embed	logistic + TF-IDF, transformer, CNN/RNN	privacy, sarcasm, context
Image	scans, photos	resize, normalize, augmentation	CNN, vision transformer	labeling cost, bias
Audio	calls, machine sound	resample, spectrogram/MFCC, normalize	CNN, RNN, transformer	noise and storage cost
Time series	daily demand, sensor signal	windowing, lag features, temporal split	AR-like baselines, trees with lags, deep sequence models	random split is usually wrong
JSON/logs	app events, clickstream	schema flattening, path extraction, count/session features	trees, boosting, sequence models	schema drift

5.5 Choosing the learning paradigm

5.5.1 Supervised learning

Use when labeled outcomes exist.

Examples: - classification: no-show vs attended - regression: demand quantity - ranking: order candidates by relevance

5.5.2 Unsupervised learning

Use when labels do not exist but structure matters.

Examples: - clustering patients by visit behavior - anomaly detection on claims - dimensionality reduction for visualization or denoising

5.5.3 Reinforcement learning

Use when actions influence future states and rewards over time.

Examples: - dynamic pricing policies - inventory control - treatment recommendation with sequential feedback - robotic control

A practical caution: many business problems that appear to be reinforcement learning are actually **supervised learning on historical outcomes**, because the action-feedback loop is not safely or reliably available.

5.6 Minimum-quality standard

A project should have, before modeling: - data inventory - row-grain definition - label source documentation - data dictionary - train/validation/test split plan - leakage checklist - permissions and privacy review

5.7 Official references

- pandas on categoricals, missing data, and time series [R5–R7]
- TensorFlow/Keras structured data preprocessing tutorials [R9–R10]
- PyTorch tutorials and torchvision datasets for custom and unstructured pipelines [R11–R13]
- Gymnasium, Stable-Baselines3, and Spinning Up for reinforcement-learning concepts and environments [R25–R30]

6 Clean

6.1 Definition

Cleaning means making the data faithful, consistent, and learnable without changing the real-world meaning. It is not "making the data look nice." It is controlled error reduction.

6.2 What to do

6.2.1 1. Resolve schema issues

Check: - column names - duplicated columns - type mismatches - inconsistent units - invalid categories - impossible values

Examples: - age = -3 - appointment duration = 0 for all rows due to extraction bug - "Male", "male", "M" stored separately

6.2.2 2. Handle missing data deliberately

Missingness is information. Distinguish: - **Missing completely at random** - **Missing at random** - **Missing not at random**

In practice, start with: - count missingness by column, - inspect whether missingness correlates with the target, - create missingness indicators when appropriate.

Common strategies: - numeric: median or model-based imputation - categorical: explicit "Unknown" bucket or most-frequent imputation - time series: forward fill only when operationally justified - text/image/audio: remove unusable records only with reason logging

6.2.3 3. Detect duplicates

Duplicates can occur at: - exact row level, - entity-time level, - label level, - near-duplicate text/image level.

6.2.4 4. Standardize categorical values

Examples: - map "Cardio", "cardiology", "CARD" to one category - normalize whitespace, casing, punctuation where appropriate

6.2.5 5. Treat outliers logically

Do not automatically delete outliers. First ask: - Is it a data-entry error? - Is it a true but rare event? - Is it exactly the kind of event the model should detect?

6.2.6 6. Build a reproducible cleaning pipeline

Cleaning logic should be code or documented transformations, not spreadsheet edits that cannot be reproduced.

6.3 Preprocessing operations by data type

Data type	Common cleaning actions	Notes
Numeric	convert units, cap impossible values, impute missing values	scaling may belong later in the pipeline
Categorical	trim text, unify spelling, map rare values	keep mapping tables versioned
Ordinal	verify true order and allowed values	incorrect ordering can distort the model
Datetime	parse formats, localize time zones, check impossible sequences	event ordering matters

Data type	Common cleaning actions	Notes
Text	remove corrupted text, normalize encoding, handle empty strings	preserve important domain tokens
Image	detect broken files, wrong color channels, unreadable images	keep image-validation report
Audio	verify sampling rate and file integrity	long silences may be meaningful
JSON/logs	flatten nested keys consistently, handle schema evolution	track source version
Time series	check gaps, duplicated timestamps, clock shifts	cleaning and resampling must respect time

6.4 Minimum-quality standard

- reproducible cleaning script or notebook promoted to pipeline code
- missingness report
- duplicates report
- invalid-value rules
- change log of dropped or corrected records
- train/validation/test split performed **before** any target-dependent transformation

6.5 Common limitations and warnings

- Imputation can hide operational problems.
- Winsorization or clipping can suppress rare but important signals.
- Aggressive text normalization can delete meaning.
- Filling future gaps in time series can cause leakage.
- Target-aware encodings must be computed inside cross-validation folds only.

6.6 Helpful official references

- pandas missing-data and type-handling documentation [R5–R7]
- scikit-learn preprocessing and compose/pipeline documentation [R2–R3]

7 Explore

7.1 Definition

Exploration means studying distributions, relationships, anomalies, and uncertainty before committing to modeling choices. It is the diagnostic stage.

7.2 What to do

7.2.1 1. Profile the target

For classification: - class balance - prevalence over time - prevalence by subgroup

For regression: - skewness - heavy tails - zero inflation - trend/seasonality over time

7.2.2 2. Profile each feature

Check: - ranges - missingness - cardinality - unusual values - drift across time slices - correlation with the target

7.2.3 3. Look for interaction and segmentation effects

The average effect may hide subgroup patterns.

Example: - reminder effectiveness may differ by appointment type - lead time may matter only for first-time patients

7.2.4 4. Use the right visual summaries

Minimum useful set: - histograms / density plots for numeric variables - count plots for categorical variables - box plots by class - correlation heatmaps for numeric variables - target rate by feature buckets - time plots for label prevalence and data volume

7.2.5 5. Write exploratory findings as hypotheses

Examples: - "Prior no-shows appear strongly predictive." - "Very long lead times may increase absence." - "Some specialties have different base rates."

The point of EDA is not to prove causality. It is to shape sensible feature and validation decisions.

7.3 Minimum-quality standard

- target profile
- feature profile
- class imbalance review
- subgroup sanity checks
- time-based drift inspection
- EDA summary written as hypotheses, not conclusions

7.4 Real-life metaphor

Exploration is like a doctor reviewing a waiting room before starting appointments. Before deciding what to do, the doctor looks around: which cases are common, which are urgent, which are incomplete, and where the hidden risk might be.

7.5 Recommended tools

- pandas for summary tables and time handling [R5–R7]
- scikit-learn inspection, preprocessing, and model-selection utilities [R1–R4]
- matplotlib or similar plotting libraries for diagnostics
- Evidently for data-quality and drift summaries in a more operational form [R23–R24]

8 Features

8.1 Definition

A feature is a measurable representation of something the model can learn from. Feature engineering is the design of useful inputs from raw data.

8.2 Core principle

Features must be: - available at prediction time, - meaningful for the decision, - stable enough to survive deployment, - transformed consistently in training and inference.

8.3 Common feature types

8.3.1 Numeric transforms

Examples: - log transforms for heavy-tailed variables - ratios - differences - rolling averages - interaction terms

8.3.2 Encoded categoricals

Common approaches: - one-hot encoding - ordinal encoding - hashing - learned embeddings - native categorical handling in CatBoost, LightGBM, and XGBoost

8.3.3 Time-derived features

Examples: - day of week - month - hour - lead time - recency - frequency - rolling counts - seasonality indicators

8.3.4 Text features

Examples: - bag of words - TF-IDF - n-grams - pretrained embeddings - transformer embeddings

8.3.5 Image and audio features

Examples: - convolutional features - spectrogram features - pretrained embeddings - data augmentations that preserve label meaning

8.4 How to identify predictive features

Use a layered approach:

1. **Domain logic**
   Ask what a skilled human would check.
2. **Temporal logic**
   Ensure the signal exists before the decision.
3. **Statistical screening**
   Use univariate summaries, mutual information, correlation, or simple baselines.
4. **Model-based importance**
   Inspect coefficients, permutation importance, or tree-based importances.
5. **Ablation testing**
   Remove groups of features and see whether performance drops.

8.5 Feature selection and dimensionality reduction

8.5.1 Feature selection

Use when: - too many features, - need interpretability, - cost of inference matters, - noisy features hurt generalization.

Methods: - filter methods: variance threshold, univariate tests, mutual information - wrapper methods: recursive feature elimination - embedded methods: L1 regularization, tree-based importance

8.5.2 Dimensionality reduction

Use when: - many correlated variables, - visualization is needed, - downstream models struggle with dimensionality.

Methods: - PCA - truncated SVD for sparse text - autoencoders - manifold methods for visualization

8.6 Encoding and scaling rules of thumb

Situation	Usually needed?	Why
Linear / logistic regression	scaling often yes	optimization stability and coefficient comparability
SVM	scaling usually yes	distance and margin sensitivity
k-NN / k-means	scaling usually yes	distance-based

Situation	Usually needed?	Why
Tree models	scaling usually no	splits are threshold-based
One-hot for low-cardinality categoricals	often yes for linear models	simple and interpretable
Native categorical handling in boosting	often preferable for tree boosting	avoids overly sparse wide matrices

8.7 Regularization

Regularization reduces overfitting by penalizing complexity.

8.7.1 L2 regularization

$$\min_{\theta} \ \mathcal{L}(\theta) + \lambda \|\theta\|_2^2$$

Effect: - shrinks weights smoothly - useful when many features share signal

8.7.2 L1 regularization

$$\min_{\theta} \ \mathcal{L}(\theta) + \lambda \|\theta\|_1$$

Effect: - can drive some weights exactly to zero - useful for sparse feature selection

8.7.3 Dropout and weight decay

Common in neural networks: - dropout randomly removes units during training - weight decay is an L2-like penalty on weights

8.8 Minimum-quality standard

- features mapped to prediction-time availability
- transformation pipeline versioned
- leakage review completed
- scaling/encoding choices justified
- feature-selection decisions documented
- feature importance or ablation analysis performed for the final shortlist

8.9 Important limitations

- Great-looking features can hide leakage.
- Importance scores are model-dependent.
- One-hot encoding can explode dimensionality.
- Target encoding can overfit if not done within folds.
- Learned embeddings can be powerful but less transparent.

8.10 Official references

- scikit-learn preprocessing, compose, feature selection, and model inspection [R2–R4]
- TensorFlow/Keras preprocessing layers for structured data [R9]
- XGBoost, LightGBM, and CatBoost official docs for native categorical support and boosting behavior [R14–R20]

9 Train

9.1 Definition

Training is the process of fitting model parameters to minimize a loss function on training data.

Formally, for a parameterized model f_θ, training solves:

$$\theta^* = \arg\min_\theta \frac{1}{n} \sum_{i=1}^{n} \mathcal{L}(y_i, f_\theta(x_i))$$

9.2 Split design comes before algorithm choice

A model can appear excellent only because the validation split is wrong. Use the split that matches deployment reality.

9.2.1 Common split strategies

Situation	Recommended split
Standard tabular IID data	train / validation / test or k-fold CV
Small labeled dataset	repeated cross-validation
Time series	chronological split / backtesting
Grouped entities	GroupKFold or group-aware split
Highly imbalanced class	stratified split
Image/text with duplicates	split by entity/source to avoid leakage
Reinforcement learning	train environment, validation seeds/scenarios, held-out test scenarios

9.3 The three major learning paradigms

9.3.1 Supervised learning

Learn from labeled examples (x_i, y_i).

Subtasks: - classification - regression - ranking - survival analysis - multi-label prediction

9.3.2 Unsupervised learning

Learn structure without labels.

Subtasks: - clustering - dimensionality reduction - density estimation - anomaly detection

9.3.3 Reinforcement learning

Learn a policy $\pi(a \mid s)$ that selects action a in state s to maximize expected cumulative reward:

$$J(\pi) = \mathbb{E}_\pi \left[\sum_{t=0}^{T} \gamma^t r_t \right]$$

where $\gamma \in [0, 1]$ is the discount factor.

9.4 Supervised algorithms: what to use, when, and why

9.4.1 Linear regression / logistic regression

Best for: - strong baseline - interpretable models - sparse text with linear decision boundaries - tabular problems with limited data

Advantages: - fast - simple - stable baseline - coefficients interpretable with care

Limitations: - weaker on complex nonlinear patterns - requires encoding of categoricals - scaling often helps - interactions must often be engineered explicitly

9.4.2 Decision trees

Best for: - quick prototype - nonlinear rules - mixed feature types

Advantages: - interpretable as rules - handles nonlinear splits - little preprocessing for numeric scaling

Limitations: - unstable - prone to overfitting - single trees are rarely state of the art

9.4.3 Random forests

Best for: - robust tabular baseline - nonlinear tabular problems - feature importance screening

Advantages: - less overfitting than a single tree - handles nonlinear relationships - usually strong baseline with minimal tuning

Limitations: - larger memory footprint - less interpretable than a single tree - probability calibration can be mediocre

9.4.4 Gradient boosting (general), XGBoost, LightGBM, CatBoost

Best for: - high-performance tabular supervised learning - nonlinear relationships - mixed numeric/categorical data - structured business problems

Advantages: - usually very strong on tabular data - handles nonlinearities and interactions well - flexible regularization and early stopping - CatBoost/LightGBM/XGBoost provide useful categorical options [R14–R20]

Limitations: - more tuning than random forests - can overfit with aggressive trees or too many rounds - category handling differs across libraries - interpretability requires additional tooling

9.4.5 Support vector machines

Best for: - small to medium datasets - high-dimensional settings - classification with margin-based separation

Advantages: - strong in some sparse/high-dimensional settings - kernel trick allows nonlinear decision boundaries

Limitations: - scaling sensitive - harder to tune - slower on very large datasets - probability outputs may require calibration

9.4.6 k-nearest neighbors

Best for: - local-pattern problems - quick nonparametric baseline - low-dimensional or well-embedded spaces

Advantages: - simple - no explicit training beyond indexing - intuitive

Limitations: - distance metric sensitive - scaling required - poor in high dimensions - expensive inference at scale

9.4.7 Neural networks / deep learning

Best for: - text, image, audio, video - large datasets - complex nonlinear relationships - transfer learning from pretrained models

Advantages: - expressive - state of the art for many unstructured-data tasks - transfer learning can reduce data needs

Limitations: - compute-intensive - more hyperparameters - less interpretable - sensitive to data volume, optimization, and monitoring design

9.5 Unsupervised algorithms

9.5.1 k-means

Best for: - fast segmentation with roughly spherical clusters

Advantages: - simple - scalable - useful first clustering baseline

Limitations: - requires k - sensitive to scale and initialization - poor on irregular cluster shapes

9.5.2 DBSCAN

Best for: - density-based clustering - outlier identification

Advantages: - does not require k - can find arbitrarily shaped clusters - identifies noise points

Limitations: - sensitive to scale and hyperparameters - weak in varying-density settings

9.5.3 Hierarchical clustering

Best for: - exploratory grouping - smaller datasets - interpretable dendrograms

Limitations: - computational cost grows quickly - linkage choice matters

9.5.4 PCA / Truncated SVD

Best for: - compression - denoising - visualization support - sparse text reduction (SVD)

Limitations: - components may be hard to interpret - unsupervised variance may not equal predictive usefulness

9.5.5 Isolation Forest / Local Outlier Factor / one-class methods

Best for: - anomaly screening

Limitations: - evaluation can be difficult without labels - contamination assumptions matter

9.6 Reinforcement learning algorithms

9.6.1 DQN

Use when: - discrete action space - value-based formulation is appropriate

Official SB3 documentation notes DQN stabilizes learning with replay buffer, target network, and gradient clipping [R28].

9.6.2 PPO

Use when: - policy optimization is desired - robust default for many simulated control tasks

SB3 documents PPO as a practical clipping-based policy-gradient method [R29].

9.6.3 SAC / DDPG / TD3

Use when: - continuous action spaces - sample-efficient off-policy learning is valuable

Spinning Up and Keras examples are good official starting points [R30–R32].

9.6.4 RL limitations that beginners often underestimate

- Requires an environment or simulator
- Rewards can be misspecified
- Results vary across random seeds
- Offline historical data does not automatically justify RL
- Evaluation is harder than in standard supervised learning

9.7 Comparative training table

Algorithm family	Typical data	Strengths	Weaknesses	Good default use
Linear / logistic	structured, sparse text	interpretable, fast, strong baseline	misses complex nonlinearity	first supervised baseline
Trees	structured	intuitive rules	overfits easily	quick prototype
Random forest	structured	robust, low-preprocessing baseline	heavier inference	nonlinear tabular baseline
Boosted trees	structured	top tabular performance	more tuning	serious tabular modeling
SVM	structured, sparse high-dimensional	strong margins	scale/tuning cost	niche small/medium problems
k-NN	structured/embedded	simple local baseline	poor scalability	sanity-check baseline
Neural nets	unstructured, large data	expressive, transfer learning	compute and tuning	text/image/audio tasks
k-means	unlabeled tabular	fast segmentation	assumes cluster geometry	exploratory clustering
DBSCAN	unlabeled tabular/spatial	finds noise and arbitrary shapes	sensitive parameters	anomaly-like segmentation
PCA/SVD	high-dimensional	compression	interpretability	preprocessing and visualization
RL methods	sequential decision problems	learns policies with delayed reward	high variance, evaluation difficulty	only when actions change future outcomes

9.8 Minimum-quality standard

- baseline model implemented
- split design justified
- reproducible training pipeline
- random seed policy documented
- class imbalance handled if relevant
- training metrics logged by run
- final model selected against validation, not test

9.9 Official references

- scikit-learn supervised, unsupervised, metrics, and model-selection guides [R1–R4]
- TensorFlow/Keras tutorials for regression, classification, and imbalanced structured data [R8–R10]
- PyTorch tutorials for deep learning workflows [R11–R12]
- XGBoost, LightGBM, CatBoost docs for boosted-tree behavior [R14–R20]
- Gymnasium, Stable-Baselines3, Keras RL examples, and Spinning Up for RL [R25–R32]

10 Evaluate

10.1 Definition

Evaluation asks: “How good is the model on unseen data, for the decision we actually care about?”

A correct metric must match: - task type, - class balance, - action costs, - threshold behavior, - deployment conditions.

10.2 Classification metrics

10.2.1 Confusion-matrix terms

For binary classification: - True Positive (TP) - False Positive (FP) - True Negative (TN) - False Negative (FN)

10.2.2 Precision

$$\text{Precision} = \frac{TP}{TP + FP}$$

Use when false positives are costly.

10.2.3 Recall

$$\text{Recall} = \frac{TP}{TP + FN}$$

Use when missed positives are costly.

10.2.4 $\mathbf{F}_1$

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Use when balancing precision and recall.

10.2.5 Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Use only when class balance and error costs make it meaningful.

10.2.6 ROC-AUC and PR-AUC

- ROC-AUC summarizes ranking quality across thresholds.
- PR-AUC is often more informative in highly imbalanced problems.

10.2.7 Log loss / cross-entropy

Useful when calibrated probabilities matter.

$$\text{LogLoss} = -\frac{1}{n} \sum_{i=1}^{n} [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$$

10.3 Regression metrics

10.3.1 Mean Absolute Error (MAE)

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^{n} |y_i - \hat{y}_i|$$

10.3.2 Mean Squared Error (MSE)

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

10.3.3 Root Mean Squared Error (RMSE)

$$\text{RMSE} = \sqrt{\text{MSE}}$$

10.3.4 Coefficient of determination R^2

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

10.4 Clustering and unsupervised evaluation

Use with care because "good structure" is context dependent.

Examples: - silhouette score - Davies–Bouldin score - stability across resamples - downstream utility - human review

10.5 Reinforcement-learning evaluation

Evaluate: - average episodic return - variance across seeds - performance on held-out scenarios - safety constraints - sample efficiency

The SB3 tips explicitly warn that RL results vary across runs and should be assessed across several seeds [R27].

10.6 Threshold selection

A model that outputs probabilities still needs a decision threshold.

Example: - low threshold → high recall, lower precision - high threshold → lower recall, higher precision

Threshold must be chosen against business cost, not convenience.

For the clinic: - if reminders are cheap and missed visits are expensive, prefer higher recall - if nurse follow-up calls are scarce, precision matters more

10.7 Calibration

Calibration means predicted probabilities should match observed frequencies.

Example: - among cases predicted at 0.80 risk, around 80% should truly be positive.

Calibration matters when probabilities drive action intensity or resource allocation.

10.8 Error analysis

Always inspect: - false positives - false negatives - subgroup performance - time-slice performance - performance by data source or site - label-noise patterns

10.9 Minimum-quality standard

- untouched test set held back until final evaluation
- metric choice justified by business cost
- confidence intervals or repeated validation where feasible
- subgroup/error analysis completed
- threshold chosen deliberately
- calibration checked if probabilities are used operationally

10.10 Official references

- scikit-learn model evaluation and metrics [R4]
- TensorFlow imbalanced-data tutorial [R10]
- Evidently for monitoring and drift-oriented evaluation views [R23–R24]

11 Tune

11.1 Definition

Tuning means improving generalization by selecting hyperparameters, architectures, thresholds, and training settings without contaminating the test set.

11.2 What to tune

11.2.1 Classical ML

Typical hyperparameters: - regularization strength - tree depth - learning rate - number of estimators - minimum samples per leaf - subsampling ratios - class weights - kernel parameters

11.2.2 Deep learning

Typical hyperparameters: - learning rate - batch size - optimizer - weight decay - dropout - architecture depth/width - augmentation policy - scheduler - early-stopping patience

11.2.3 Reinforcement learning

Typical hyperparameters: - learning rate - discount factor γ - rollout length - replay buffer size - entropy coefficient - clipping range - network architecture

11.3 Search methods

11.3.1 Grid search

Pros: - simple - exhaustive over a small grid

Cons: - inefficient in large spaces

11.3.2 Random search

Pros: - often better than grid in wide spaces - easy to parallelize

Cons: - can miss narrow optima

11.3.3 Bayesian optimization / model-based search

Pros: - more sample efficient for expensive training

Cons: - extra complexity - assumptions about the search surface

11.3.4 Early-stopping schedulers

Examples: - ASHA / HyperBand-style approaches - pruning unpromising runs

Useful when training is expensive.

11.4 Practical tuning rules

1. Start with a strong baseline.
2. Tune the most sensitive hyperparameters first.
3. Use cross-validation or repeated validation if data is small.
4. Keep the test set untouched.

5. Log every run.
6. Prefer coarse-to-fine search.
7. Use early stopping where supported.
8. Tune the **decision threshold** separately from the model.

11.5 Tuning by algorithm family

Family	High-impact hyperparameters
Logistic / linear	regularization strength, penalty type, feature scaling, class weights
Random forest	number of trees, max depth, max features, min samples leaf
Boosting	learning rate, number of estimators, depth/leaves, subsampling, regularization
SVM	C, kernel, γ
Neural nets	learning rate, batch size, architecture, dropout, optimizer, weight decay
RL	learning rate, discount factor, rollout/replay settings, entropy/clipping, network size

11.6 Minimum-quality standard

- search space justified
- experiment tracking enabled
- validation protocol fixed before tuning
- early stopping or budget control defined
- best model selected on validation only
- final performance reported once on test

11.7 Official references

- scikit-learn hyperparameter tuning utilities [R3]
- Ray Tune search-space and tuning docs [R33–R36]
- PyTorch and TensorFlow tutorials for training-loop integration [R11, R34]
- Stable-Baselines3 RL tuning guidance [R27]

12 Deploy

12.1 Definition

Deployment means making the trained model available for real use. This usually includes the model artifact, preprocessing logic, runtime environment, API or batch interface, versioning, and rollback capability.

12.2 Deployment patterns

12.2.1 Batch scoring

Use when: - decisions can be made hourly, daily, or weekly - low latency is not required

Examples: - nightly risk scores - daily demand forecasts

12.2.2 Online API inference

Use when: - predictions are needed in near real time - user or system actions depend immediately on output

Examples: - fraud screening - call-center routing - personalization

12.2.3 Embedded / edge inference

Use when: - latency, privacy, or connectivity constraints require local inference

12.3 What to package

At minimum, package: - trained model - preprocessing pipeline - label mapping - feature-order schema - dependency versions - environment configuration - model card / release notes - inference test cases

12.4 Serialization and portability

scikit-learn's model-persistence guide summarizes several options, including ONNX and skops, and warns about security and maintainability tradeoffs of pickle-like methods [R21–R22].

12.4.1 Practical rule

- Prefer portable or safer formats where possible.
- Persist the entire preprocessing + model pipeline, not just the estimator.

12.5 Serving stack

A common and reasonable stack is: - model artifact - Python service (for example FastAPI) - container image (Docker) - orchestration (optional Kubernetes) - model registry / tracking (MLflow)

Official references: - FastAPI deployment guidance [R37–R39] - Docker Python/container guides [R40–R42] - Kubernetes deployment concepts [R43–R45] - MLflow tracking, registry, and serving [R46–R49]

12.6 Deployment checklist

Area	Minimum standard
Reproducibility	exact code version, dependency lock, artifact version
Inputs	schema validation and required fields
Outputs	stable response contract and label semantics
Security	secret management, no unsafe deserialization exposed
Testing	unit, integration, and shadow/batch reconciliation tests
Operations	logging, latency, error rate, rollback plan
Governance	approval record, model card, owner

12.7 Common deployment mistakes

- Training and inference preprocessing differ
- Feature order changes silently
- Model file loads but schema is wrong
- No fallback if the service fails
- No versioning or rollback
- No monitoring hook for prediction drift

12.8 Clinic example

A practical deployment could be: 1. score tomorrow's appointments each evening, 2. write risk score and risk band back to scheduling table, 3. trigger reminder workflow for high-risk cases, 4. log predictions and outcomes for later monitoring.

13 Monitor

13.1 Definition

Monitoring is the ongoing measurement of whether the data, predictions, and outcomes remain healthy after deployment.

A model can fail even when the code still runs.

13.2 The four monitoring layers

13.2.1 1. System monitoring

Track: - latency - throughput - error rate - CPU/GPU/memory - queue depth - uptime

13.2.2 2. Data-quality monitoring

Track: - missing fields - schema violations - range violations - category explosions - null spikes - join failure rates

Great Expectations provides official workflows for validating data and creating validation results and data docs [R50–R52].

13.2.3 3. Drift monitoring

Track: - input feature drift - prediction drift - target drift where labels arrive later

Evidently documents data-drift reporting and practical checks for data and ML systems [R23–R24].

13.2.4 4. Performance monitoring

Track: - precision / recall / MAE / RMSE / R^2 or relevant task metrics once labels arrive - calibration - subgroup fairness gaps - business KPI impact

13.3 What to compare against

Use one or more references: - training baseline - recent stable production window - gold-validation dataset - canary population - subgroup-specific baselines

13.4 Alert design

Do not alert on everything. Alert on: - conditions that require action, - thresholds tied to real risk, - persistent deviations rather than single noise spikes.

13.5 Retraining triggers

Examples: - accuracy drop beyond threshold - drift beyond tolerance - new categories exceed threshold - policy/regulation change - seasonal behavior shift - new upstream data source deployed

13.6 Minimum-quality standard

- prediction logging with identifiers and timestamps
- data-quality checks automated
- drift report automated
- delayed-label performance review defined
- alert thresholds documented
- retraining policy defined
- rollback or disable switch available

13.7 Practical limitation

Not every model needs continuous retraining. Some need only periodic review. The correct cadence depends on how quickly the world changes.

14 Repeat

14.1 Definition

Repeat means the workflow is cyclical, not linear. Every production model changes the problem environment or reveals new constraints.

14.2 What to revisit

- target validity
- data-contract changes
- new subgroups
- action policy changes
- calibration drift
- fairness concerns
- model simplification opportunities
- baseline refresh

14.3 Post-deployment review questions

1. Did the model improve the business process?
2. Which errors caused the most operational pain?
3. Which features became unreliable?
4. Did users trust and use the outputs?
5. Is a simpler model now sufficient?
6. Has the target itself changed?

14.4 Improvement paths

- better labels
- better temporal validation
- richer features
- better intervention design
- recalibration
- model replacement
- full retirement

15 A Minimum Standard Workflow Blueprint

This section condenses the full guide into an operational blueprint.

15.1 Phase 1: Scope correctly

- write the decision statement
- define target, row grain, prediction time
- state the intervention and business metric
- define constraints and risks

15.2 Phase 2: Build trustworthy data

- inventory sources

- create data dictionary
- define split strategy
- check leakage and permissions

15.3 Phase 3: Prepare the learning substrate

- clean types, missingness, categories, duplicates
- explore distributions and target balance
- engineer prediction-time-safe features
- build reproducible preprocessing pipelines

15.4 Phase 4: Model systematically

- start with interpretable/simple baseline
- add stronger algorithm families
- evaluate with correct metrics
- tune with tracked experiments
- preserve untouched test set

15.5 Phase 5: Productionize responsibly

- package preprocessing + model together
- validate schema on input
- deploy with versioning and rollback
- monitor system, data, drift, and performance
- retrain only when triggers justify it

16 Algorithm and Technique Selection Cheat Sheets

16.1 Supervised learning cheat sheet

Scenario	Recommended first-line options	Why
Small tabular classification	logistic regression, random forest, gradient boosting	good baseline range from simple to nonlinear
Large tabular classification	LightGBM / XGBoost / CatBoost	strong performance and scalability
Sparse text classification	logistic regression with TF-IDF, linear SVM	strong, fast, interpretable baseline
Image classification	transfer learning with PyTorch or Keras	pretrained models reduce data demand
Regression with explainability needs	linear regression, GAM-like approaches, gradient boosting with interpretation	balances accuracy and transparency
Very imbalanced classification	class weighting, calibrated boosting, threshold optimization	default accuracy is misleading

16.2 Unsupervised learning cheat sheet

Goal	Good first method	Check before trusting results
customer / patient segmentation	k-means	scale variables, compare several k, inspect stability
outlier screening	Isolation Forest or DBSCAN	define what an actionable anomaly means

Goal	Good first method	Check before trusting results
compressed visualization	PCA / SVD	variance explained and business interpretability
topic-like text grouping	embeddings + clustering	verify cluster meaning manually

16.3 Reinforcement learning cheat sheet

Situation	Likely fit	Notes
discrete actions in simulation	DQN / PPO	start with stable implementations
continuous control	SAC / DDPG / TD3 / PPO	evaluate over multiple seeds
business process with weak simulator	often **not** ready for RL	start with supervised/offline evaluation instead

17 Teaching Notes for Trainers

17.1 How to explain the workflow simply

- **Problem** = deciding what job the model is hired to do
- **Data** = the experience the model learns from
- **Clean** = fixing messy paperwork
- **Explore** = looking around before making assumptions
- **Features** = turning raw clues into learnable signals
- **Train** = practice
- **Evaluate** = exam on unseen cases
- **Tune** = refining how the learner studies
- **Deploy** = putting the learner on shift
- **Monitor** = supervision after onboarding
- **Repeat** = continuous professional development

17.2 Suggested classroom sequence

1. One business problem
2. One dataset
3. One baseline model
4. One stronger model
5. One validation design
6. One deployment mock-up
7. One monitoring dashboard concept

This sequencing prevents beginners from drowning in algorithm names.

18 Conclusion

The end-to-end workflow matters more than any single algorithm. For most real projects, disciplined problem framing, leakage prevention, correct validation, and post-deployment monitoring create more value than a marginal modeling improvement. A practical learning path is therefore:

1. master tabular supervised learning first,
2. learn pipelines and validation deeply,
3. add boosted trees and basic deep learning,
4. then study deployment and monitoring,
5. and only then move into more specialized areas such as reinforcement learning or advanced representation learning.

19 Appendix A. Exhaustive Glossary of Technical Terms

19.1 A

Ablation study
A controlled test in which you remove one feature, component, or design choice at a time to see how much it mattered.

Accuracy
The proportion of all predictions that are correct. Useful only when class balance and error costs are appropriate.

Action policy
In reinforcement learning, the rule that selects an action for each state.

API (Application Programming Interface)
A defined way for one software system to request services from another. For model serving, an API usually receives inputs and returns predictions.

Artifact
A saved output of the modeling process, such as a trained model, feature map, or evaluation report.

19.2 B

Bagging
An ensemble strategy that trains multiple models on resampled datasets and aggregates their outputs. Random forest is a classic example.

Baseline model
A simple reference model used to set a minimum acceptable benchmark.

Batch inference
Generating predictions for many records together on a schedule rather than one request at a time.

Bias (statistical)
Systematic error caused by a model being too simple or assumptions being wrong.

Boosting
An ensemble strategy that builds models sequentially so later models focus on earlier mistakes.

19.3 C

Calibration
Agreement between predicted probabilities and observed frequencies.

Categorical feature
A variable whose values are labels or groups rather than measured quantities.

Class imbalance
A situation in which one class is much rarer than another.

Classification
A supervised task in which the output is a category.

Concept drift
A change in the relationship between inputs and target over time.

Cross-validation
A validation procedure that rotates training and validation folds to estimate generalization.

19.4 D

Data drift
A change in the input data distribution over time.

Data leakage
Use of information during training that would not be available at prediction time.

Decision threshold
The probability cut-off used to convert model scores into discrete actions.

Deep learning
Machine learning using multi-layer neural networks.

Dimensionality reduction
Compressing many variables into fewer derived variables.

Distribution
The pattern of how values are spread across a variable.

19.5 E

Early stopping
Halting training when validation performance stops improving.

Embedding
A learned dense numerical representation of data such as words, categories, or images.

Ensemble
A combination of multiple models.

Epoch
One full pass through the training dataset in neural-network training.

Evaluation metric
A numerical measure used to judge model performance.

19.6 F

False negative
A positive case predicted as negative.

False positive
A negative case predicted as positive.

Feature
An input variable used by the model.

Feature engineering
Creating or transforming inputs to make patterns easier to learn.

Feature importance
A measure of how influential a feature is to predictions, under a given method.

F_1 score
The harmonic mean of precision and recall.

19.7 G

Generalization
How well a model performs on unseen data.

Gradient
The direction and magnitude of steepest change in a function; used in optimization.

Gradient boosting
A boosting approach that adds models to reduce residual errors.

19.8 H

Hyperparameter
A configuration chosen before or during training, such as learning rate or tree depth.

Hyperparameter tuning
Searching for hyperparameter values that improve validation performance.

19.9 I

IID (Independent and identically distributed)
An assumption that observations are independent draws from the same distribution.

Imputation
Filling in missing values using a chosen rule or model.

Inference
Using a trained model to make predictions.

Interpretability
How understandable a model or its outputs are to humans.

19.10 K

k-fold cross-validation
A cross-validation method that splits the data into k folds and rotates the validation fold.

k-means
A clustering algorithm that partitions data into k clusters.

k-nearest neighbors (k-NN)
A method that predicts using nearby training examples.

19.11 L

Label
The true outcome the model is trained to predict.

Learning rate
A training-control parameter that determines step size during optimization.

Linear model
A model that forms predictions as a weighted linear combination of inputs.

Log loss
A probability-sensitive classification loss based on cross-entropy.

19.12 M

MAE
Mean Absolute Error; average absolute deviation between truth and prediction.

MSE
Mean Squared Error; average squared deviation between truth and prediction.

Missingness indicator
A feature showing whether an original value was missing.

Model card
A structured summary of model purpose, data, evaluation, risks, and limitations.

Model drift
A broad term for deterioration in model usefulness due to environmental or data change.

19.13 N

Neural network
A parameterized function made of layers of connected units.

Normalization
A broad term for rescaling or standardizing data.

19.14 O

Objective function
The function being optimized during training, such as loss plus regularization.

One-hot encoding
Representing a category as a binary vector with one active position.

Overfitting
Learning training-specific noise instead of generalizable pattern.

19.15 P

Pipeline
A reproducible chain of transformations and model steps.

Precision
Of the cases predicted positive, the proportion truly positive.

Preprocessing
All transformations needed to turn raw data into model-ready inputs.

Probability calibration
Adjusting or checking whether predicted probabilities are numerically reliable.

19.16 Q

Quality gate
A checkpoint condition that must be met before proceeding.

19.17 R

Recall
Of the truly positive cases, the proportion correctly detected.

Regularization
A penalty or constraint used to reduce overfitting.

Regression
A supervised task in which the output is numeric.

Reinforcement learning
Learning by trial and error through actions, states, and rewards over time.

RMSE
Root Mean Squared Error.

R^2
Coefficient of determination; proportionate improvement over predicting the mean in regression.

19.18 S

Scaling
Resizing variables to comparable ranges or distributions.

Schema
The structural definition of data fields and types.

Semi-structured data
Data with some organization but flexible or nested structure, such as JSON.

Serving
Making predictions available in a running system.

Sparse data
Data in which most entries are zero or absent.

Split
A partition of the data into train, validation, and/or test subsets.

Standardization
Transforming a variable to zero mean and unit variance, often $z = (x - \mu)/\sigma$.

Supervised learning
Learning from labeled examples.

19.19 T

Target
The specific outcome variable being predicted.

Temporal leakage
Leakage caused by using future information for past predictions.

Test set
A held-out dataset used only for final evaluation.

TF-IDF
Term Frequency–Inverse Document Frequency, a classic text-vectorization method.

Threshold
The score cut-off used to trigger a class or action.

Time series
Data ordered in time where order matters.

Train/validation/test split
A three-way split used for fitting, tuning, and final evaluation.

19.20 U

Underfitting
Failing to learn enough structure from the data.

Unsupervised learning
Learning without explicit labels.

19.21 V

Validation set
Data used to compare models or tune hyperparameters.

Variance (model)
Sensitivity of a model to fluctuations in the training data.

Vectorization
Converting raw data such as text into numerical representation.

Versioning
Tracking distinct states of code, data, features, or models.

19.22 W

Weight decay
A regularization technique commonly implemented like an L2 penalty in deep learning.

Windowing
Creating time-based slices or rolling contexts from sequential data.

20 Appendix B. Python Code Templates by Workflow Section

20.1 B1. Problem framing skeleton

```
project = {
    "decision_statement": (
        "Predict no-show risk 24h before appointment "
        "so staff can prioritize reminders."
    ),
    "prediction_unit": "appointment",
    "target_name": "no_show",
    "decision_time": "24h_before_visit",
    "business_metric": "reduced_no_show_rate",
    "model_metric": "recall_at_precision_threshold",
    "constraints": [
        "PII handling",
        "latency < 1s for API or overnight batch",
        "auditable features",
    ],
}
```

20.2 B2. Data audit with pandas

```
import pandas as pd

df = pd.read_csv("appointments.csv", parse_dates=["booking_time", "appointment_time"])

audit = pd.DataFrame({
    "dtype": df.dtypes.astype(str),
    "missing_pct": df.isna().mean().round(4),
    "n_unique": df.nunique(dropna=False)
}).sort_values("missing_pct", ascending=False)

print(audit)
```

20.3 B3. Reproducible preprocessing pipeline for structured data

```
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression

numeric_features = ["age", "lead_time_days", "prior_no_show_count", "distance_km"]
categorical_features = ["appointment_type", "clinic_id", "day_of_week"]

numeric_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
])

categorical_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore")),
```

```
])

preprocess = ColumnTransformer([
    ("num", numeric_pipe, numeric_features),
    ("cat", categorical_pipe, categorical_features),
])

model = Pipeline([
    ("preprocess", preprocess),
    ("clf", LogisticRegression(max_iter=1000, class_weight="balanced"))
])

X = df[numeric_features + categorical_features]
y = df["no_show"]
model.fit(X, y)
```

20.4 B4. Strong tabular baseline with gradient boosting

```
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)

gb = Pipeline([
    ("preprocess", preprocess),
    ("clf", HistGradientBoostingClassifier(random_state=42))
])

gb.fit(X_train, y_train)
```

20.5 B5. Cross-validation and evaluation

```
from sklearn.model_selection import StratifiedKFold, cross_validate

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_validate(
    model,
    X,
    y,
    cv=cv,
    scoring=["accuracy", "precision", "recall", "f1", "roc_auc"],
    return_train_score=False
)

for metric, values in scores.items():
    if metric.startswith("test_"):
        print(metric, values.mean().round(4), values.std().round(4))
```

20.6 B6. Threshold tuning

```
import numpy as np
from sklearn.metrics import precision_recall_curve
```

```
model.fit(X_train, y_train)
proba = model.predict_proba(X_test)[:, 1]

precision, recall, thresholds = precision_recall_curve(y_test, proba)

# example rule: choose highest recall subject to precision >= 0.30
valid = np.where(precision[:-1] >= 0.30)[0]
best_idx = valid[np.argmax(recall[:-1][valid])]
best_threshold = thresholds[best_idx]

print("Selected threshold:", best_threshold)
```

20.7 B7. Hyperparameter tuning with scikit-learn

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import loguniform

param_grid = {
    "clf__C": loguniform(1e-3, 1e2)
}

search = RandomizedSearchCV(
    estimator=model,
    param_distributions=param_grid,
    n_iter=20,
    scoring="roc_auc",
    cv=cv,
    random_state=42,
    n_jobs=-1
)

search.fit(X_train, y_train)
print(search.best_params_, search.best_score_)
```

20.8 B8. Experiment tracking with MLflow

```
import mlflow
import mlflow.sklearn

with mlflow.start_run(run_name="logreg_baseline"):
    model.fit(X_train, y_train)
    auc = cross_validate(
        model, X_train, y_train, cv=cv, scoring=["roc_auc"]
    )["test_roc_auc"].mean()

    mlflow.log_param("model_family", "logistic_regression")
    mlflow.log_metric("cv_roc_auc", float(auc))
    mlflow.sklearn.log_model(model, artifact_path="model")
```

20.9 B9. Export model to ONNX (illustrative)

```
# illustrative example; requires sklearn-onnx and compatible preprocessing
# from skl2onnx import convert_sklearn
# from skl2onnx.common.data_types import FloatTensorType
#
```

```
# onnx_model = convert_sklearn(
#     model,
#     initial_types=[("input", FloatTensorType([None, len(feature_cols)]))],
# )
# with open("model.onnx", "wb") as f:
#     f.write(onnx_model.SerializeToString())
```

20.10 B10. Minimal FastAPI serving skeleton

```
from fastapi import FastAPI
from pydantic import BaseModel
import joblib
import pandas as pd

app = FastAPI()
model = joblib.load("model.joblib")

class AppointmentRequest(BaseModel):
    age: float
    lead_time_days: float
    prior_no_show_count: float
    distance_km: float
    appointment_type: str
    clinic_id: str
    day_of_week: str

@app.post("/predict")
def predict(req: AppointmentRequest):
    X = pd.DataFrame([req.model_dump()])
    p = float(model.predict_proba(X)[:, 1][0])
    return {"no_show_probability": p}
```

20.11 B11. Data-quality validation idea

```
# Example checks you might implement with Great Expectations or custom validation
expected_columns = {
    "age", "lead_time_days", "prior_no_show_count",
    "distance_km", "appointment_type", "clinic_id", "day_of_week"
}

assert set(X.columns) == expected_columns
assert X["lead_time_days"].ge(0).all()
assert X["age"].between(0, 120).all()
```

20.12 B12. Drift monitoring idea with Evidently

```
# illustrative example
# from evidently import Report
# from evidently.presets import DataDriftPreset
#
# report = Report(metrics=[DataDriftPreset()])
# report.run(reference_data=reference_df, current_data=current_df)
# report.save_html("drift_report.html")
```

20.13 B13. Text classification baseline

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression

text_model = Pipeline([
    ("tfidf", TfidfVectorizer(ngram_range=(1, 2), min_df=3)),
    ("clf", LogisticRegression(max_iter=1000))
])

text_model.fit(text_train, y_train)
```

20.14 B14. Keras structured-data baseline

```
import tensorflow as tf

model = tf.keras.Sequential([
    tf.keras.layers.Dense(64, activation="relu"),
    tf.keras.layers.Dense(32, activation="relu"),
    tf.keras.layers.Dense(1, activation="sigmoid")
])

model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=[
        tf.keras.metrics.AUC(name="auc"),
        tf.keras.metrics.Precision(),
        tf.keras.metrics.Recall(),
    ]
)

model.fit(train_ds, validation_data=val_ds, epochs=20)
```

20.15 B15. PyTorch image-classification skeleton

```
import torch
from torch import nn
from torch.utils.data import DataLoader
from torchvision import datasets, transforms, models

transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
])

train_ds = datasets.ImageFolder("train_images", transform=transform)
train_loader = DataLoader(train_ds, batch_size=32, shuffle=True)

model = models.resnet18(weights="DEFAULT")
model.fc = nn.Linear(model.fc.in_features, 2)

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)
```

```
for epoch in range(5):
    model.train()
    for X_batch, y_batch in train_loader:
        optimizer.zero_grad()
        logits = model(X_batch)
        loss = criterion(logits, y_batch)
        loss.backward()
        optimizer.step()
```

20.16 B16. Stable-Baselines3 PPO example

```
import gymnasium as gym
from stable_baselines3 import PPO

env = gym.make("CartPole-v1")
model = PPO("MlpPolicy", env, verbose=1)
model.learn(total_timesteps=50_000)
```

20.17 B17. A compact end-to-end checklist object

```
quality_gate = {
    "problem_defined": True,
    "row_grain_defined": True,
    "leakage_reviewed": True,
    "split_strategy_documented": True,
    "baseline_trained": True,
    "test_set_untouched": True,
    "tracking_enabled": True,
    "deployment_contract_defined": True,
    "monitoring_plan_defined": True
}
```

21 References

Primary and official sources only

- **[R1]** scikit-learn User Guide. https://scikit-learn.org/stable/user_guide.html
- **[R2]** scikit-learn: Preprocessing data. https://scikit-learn.org/stable/modules/preprocessing.html
- **[R3]** scikit-learn: Model selection and evaluation. https://scikit-learn.org/stable/modules/grid_search.html
- **[R4]** scikit-learn: Metrics and scoring. https://scikit-learn.org/stable/modules/model_evaluation.html
- **[R5]** pandas: Categorical data. https://pandas.pydata.org/docs/user_guide/categorical.html
- **[R6]** pandas: Working with missing data. https://pandas.pydata.org/pandas-docs/stable/user_guide/missing_data.html
- **[R7]** pandas: Time series / date functionality. https://pandas.pydata.org/pandas-docs/stable/user_guide/timeseries.html
- **[R8]** TensorFlow/Keras: Basic regression tutorial. https://www.tensorflow.org/tutorials/keras/regression
- **[R9]** TensorFlow/Keras: Classify structured data using Keras preprocessing layers. https://www.tensorflow.org/tutorials/structured_data/preprocessing_layers
- **[R10]** TensorFlow/Keras: Classification on imbalanced data. https://www.tensorflow.org/tutorials/structured_data/imbalanced_data
- **[R11]** PyTorch Tutorials Index. https://docs.pytorch.org/tutorials/index.html
- **[R12]** PyTorch: Learning with examples. https://docs.pytorch.org/tutorials/beginner/pytorch_with_examples.html
- **[R13]** Torchvision datasets documentation. https://docs.pytorch.org/vision/main/datasets.html
- **[R14]** XGBoost Documentation. https://xgboost.readthedocs.io/
- **[R15]** XGBoost Parameters. https://xgboost.readthedocs.io/en/stable/parameter.html
- **[R16]** XGBoost categorical-data tutorial. https://xgboost.readthedocs.io/en/stable/tutorials/categorical.html
- **[R17]** LightGBM Features. https://lightgbm.readthedocs.io/en/latest/Features.html
- **[R18]** LightGBM Advanced Topics. https://lightgbm.readthedocs.io/en/latest/Advanced-Topics.html
- **[R19]** CatBoost documentation. https://catboost.ai/docs/en/
- **[R20]** CatBoost categorical features. https://catboost.ai/docs/en/features/categorical-features
- **[R21]** scikit-learn Model persistence. https://scikit-learn.org/stable/model_persistence.html
- **[R22]** ONNX Introduction. https://onnx.ai/onnx/intro/
- **[R23]** Evidently introduction. https://docs.evidentlyai.com/introduction
- **[R24]** Evidently data drift. https://docs.evidentlyai.com/metrics/preset_data_drift
- **[R25]** Gymnasium introduction and basic usage. https://gymnasium.farama.org/introduction/basic_usage/
- **[R26]** Gymnasium API: Env. https://gymnasium.farama.org/api/env/
- **[R27]** Stable-Baselines3: Reinforcement learning tips and tricks. https://stable-baselines3.readthedocs.io/en/master/guide/rl_tips.html
- **[R28]** Stable-Baselines3: DQN. https://stable-baselines3.readthedocs.io/en/master/modules/dqn.html
- **[R29]** Stable-Baselines3: PPO. https://stable-baselines3.readthedocs.io/en/master/modules/ppo.html
- **[R30]** OpenAI Spinning Up: Key concepts in RL. https://spinningup.openai.com/en/latest/spinningup/rl_intro.html
- **[R31]** Keras reinforcement-learning examples. https://keras.io/examples/rl/
- **[R32]** OpenAI Spinning Up: Soft Actor-Critic. https://spinningup.openai.com/en/latest/algorithms/sac.html
- **[R33]** Ray Tune overview. https://docs.ray.io/en/latest/tune/index.html
- **[R34]** Ray Tune search spaces. https://docs.ray.io/en/latest/tune/tutorials/tune-search-spaces.html
- **[R35]** Ray Tune search space API. https://docs.ray.io/en/latest/tune/api/search_space.html
- **[R36]** Ray Tune FAQ. https://docs.ray.io/en/latest/tune/faq.html
- **[R37]** FastAPI deployment. https://fastapi.tiangolo.com/deployment/
- **[R38]** FastAPI deployment concepts. https://fastapi.tiangolo.com/deployment/concepts/
- **[R39]** FastAPI in containers. https://fastapi.tiangolo.com/deployment/docker/
- **[R40]** Docker: Containerize a Python application. https://docs.docker.com/guides/python/containerize/

- **[R41]** Docker: What is a container? https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/
- **[R42]** Docker Python guide. https://docs.docker.com/guides/python/
- **[R43]** Kubernetes overview. https://kubernetes.io/docs/concepts/overview/
- **[R44]** Kubernetes deployments. https://kubernetes.io/docs/concepts/workloads/controllers/deployment/
- **[R45]** Kubernetes tutorial: deploy an app. https://kubernetes.io/docs/tutorials/kubernetes-basics/deploy-app/deploy-intro/
- **[R46]** MLflow documentation. https://mlflow.org/docs/latest/
- **[R47]** MLflow Tracking. https://mlflow.org/docs/latest/ml/tracking/
- **[R48]** MLflow Model Registry. https://mlflow.org/docs/latest/ml/model-registry/
- **[R49]** MLflow Serving. https://mlflow.org/docs/latest/ml/deployment/
- **[R50]** Great Expectations: Validate Data. https://docs.greatexpectations.io/docs/0.18/oss/guides/validation/validate_data_lp
- **[R51]** Great Expectations: Data Validation workflow. https://docs.greatexpectations.io/docs/0.18/oss/guides/validation/validate_data_overview
- **[R52]** Great Expectations: Data Docs. https://docs.greatexpectations.io/docs/0.18/reference/learn/terms/data_docs